시스템 프로그래밍

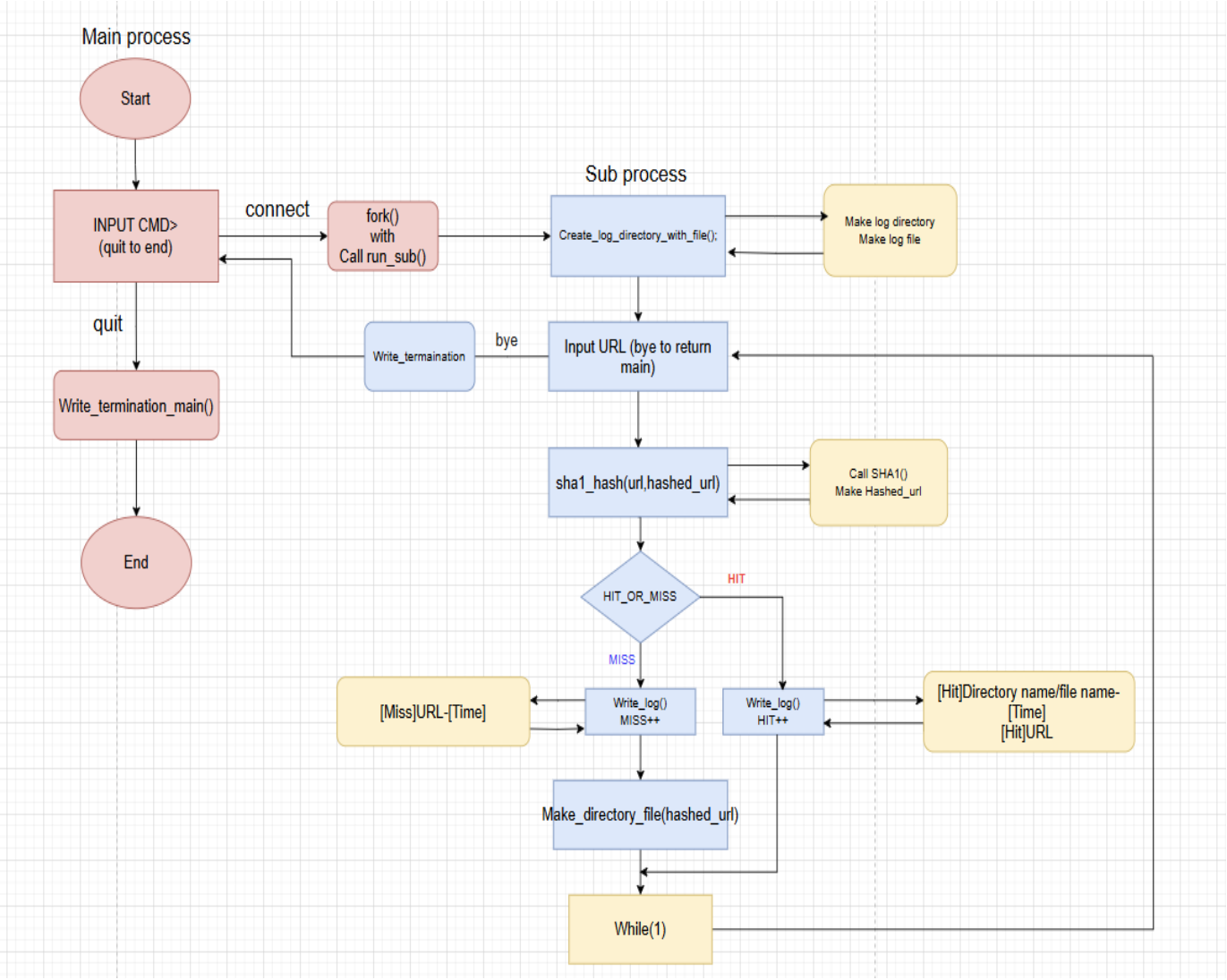
Proxy 1-3

강의 시간	목4 (B)
담당 교수님	최상호 교수님
학번	2021202089
이름	오나균

1) Introduction

Proxy 1-3 과제는 이전에 구현한 Proxy 1-2에 이어서, 추가적인 기능을 구현하는 것입니다. 이번 과제에서 구현할 기능은 우선 메인 프로세서에서 connect, quit 둘 중 하나의 CMD를 입력받습니다. connect를 입력받으면, 서브 프로세서가 생성되고, 이제 그 서브 프로세서에서 이전까지 구현한, Proxy 1-2에서의 기능을 수행하게 됩니다. 마찬가지로 서브 프로세서에서 bye를 입력받게되면, 서브 프로세서가 종료되고, 메인 프로세서로 돌아오게 됩니다. 그리고 만약 메인 프로세서에서 quit를 입력받으면, 동작 시간과 생성한 서브 프로세서의 수를 logfile/logfile.txt에 기록해주며 프로세서를 종료합니다. 지금까지 설명한 내용을 구체적으로 정리하자면, main() 함수는 메인 프로세스에서 명령어 입력을 처리하는 루프를 통해 동작시키게 하며, connect 명령어 입력 시 fork() 함수를 이용하여 자식 프로세스를 생성하고, 해당 프로세스에서 Proxy 1-2의 기능을 수행하도록 설계하면 될 것 같습니다.

2) Flow chart



3) Pseudo code

- main()

시작 시간 저장

자식 프로세스 수 초기화

```
While(1) {
사용자로부터 명령어 입력 받기 (input_cmd)
명령어가 "connect"이면:
 자식 프로세스 생성 (fork)
 자식 프로세스에서는:
     run_sub() 호출
     종료
 부모 프로세스에서는:
     자식 프로세스 수 증가
     자식 프로세스 종료 대기
명령어가 "quit"이면:
     반복 종료
}
```

종료 시간 저장 및 전체 실행 시간 계산

계산한 실행시간과 자식 프로세서 수에 대한 서버 종료 메시지 출력

프로세서 종료

- **run_sub()**

hit, miss 카운터 초기화

시작 시간 저장

로그 디렉토리와 로그 파일 생성

bye 입력받기전 까지 무한반복 :

사용자로부터 URL 입력 받기

입력이 "bye"이면 반복 종료

URL을 SHA1 해시로 변환 → hashed_url

hashed_url이 캐시에 존재하는지 확인:

존재하면 (HIT) :

hit 카운터 증가

로그 파일에 HIT 기록

존재하지 않으면 (MISS) :

miss 카운터 증가

로그 파일에 MISS 기록

캐시 디렉토리 및 파일 생성

}

종료 시간 저장 및 실행 시간 계산

hit/miss 통계 로그에 기록

4) 결과화면

```
kw2021202089@ubuntu:~$ ./proxy_cache
[2397]input CMD> connect
[2398]input url>www.kw.ac.kr
[2398]input url>www.google.com
[2398]input url>bye
[2397]input CMD> connect
[2402]input url>www.kw.ac.kr
[2402]input url>www.naver.com
[2402]input url>bye
[2397]input CMD> quit
kw2021202089@ubuntu:~$ cat ~/logfile/logfile.txt
[Miss]www.kw.ac.kr-[2025/4/14, 19:32:40]
[Miss]www.google.com-[2025/4/14, 19:32:42]
[Terminated] run time: 6 sec. #request hit : 0, miss : 2
[Hit]e00/0f293fe62e97369e4b716bb3e78fababf8f90-[2025/4/14, 19:32:46]
[Hit]www.kw.ac.kr
[Miss]www.naver.com-[2025/4/14, 19:32:48]
[Terminated] run time: 5 sec. #request hit : 1, miss : 1
 **SERVER** [Terminated] run time: 16 sec. #sub process: 2
kw2021202089@ubuntu:~$
```

- ⇨ **예제와 마찬가지로 진행**
- ⇨ **connect 입력**
- ⇨ **Sub process에서 proxy 1-2 기능 수행**
- ⇨ **connect 입력**
- ⇨ **Sub process에서 proxy 1-2 기능 수행**
- ⇨ **quit**
- ⇨ **cat ~/logfile/logfile.txt를 통해 logfile.txt 파일에 기록된 내용확인**

5) 고찰

이번 과제는 이전에 구현한 Proxy 1-2에 이어서, 사용자 요청 처리를 위해서, 메인 프로세서에서는 CMD (connect, quit)의 명령어를 입력받게하고, 이어서 새로운 자식 프로세서를 생성한 뒤, 자식 프로세서에서는 이전에 구현한 Proxy 1-2의 기능을 그대로 수행하게 하게끔 역할을 분리시키는 과제였습니다.

처음에는 프로그램 안에서 프로세서를 나눠서 부모 프로세서와 자식 프로세서를 할당한다는 개념이 낯설었는데, 개념 학습을 다시 해보니까, 리눅스에서는 fork() 함수를 사용해서, fork()가 0으로 할당되게 되면 자식 프로세서를 생성하고, 이 생성된 자식 프로세서에서 처리를 하고, 부모 프로세서는 wait()함수를 사용하여, 동작중인 자식 프로세서를 기다릴 수 있다는 것을 알게 되었습니다.

이번 과제를 진행하면서 logfile.txt 파일에 HIT 로그를 기록하는 과정에서, 깨진 문자가 무한히 출력되는 오류가 발생하였습니다. 문제의 원인은 hit과 miss를 전역 변수로 선언해 놓았기 때문이었습니다.

이로 인해 이전 서브 프로세스에서 계산된 hit 값이 이후 프로세스에도 남아 있었고, 로그 기록 조건문인 if (hit이면)이 항상 참으로 평가되어, 유효하지 않은 hashed_url 값으로 로그가 반복해서 출력되는 현상이 발생하였습니다.

이를 해결하기 위해 hit과 miss 변수를 run_sub() 함수의 지역 변수로 변경하여, 각 프로세스 내에서만 값을 유지하도록 수정하였습니다. 또한, HIT 로그를 작성하는 if문에 hashed_url != NULL 조건을 추가하여, 보다 안전하게 동작하도록 이중으로 체크하였습니다.

6) Reference

https://kim-hoya.tistory.com/7 **(fork함수 사용법)**

https://reakwon.tistory.com/45 **(wait 함수 사용법)**